Version 0.2.5

Presentate

Simple dynamic presentation in **Typst**,
with flexible package integration framework.

User Manual

GitHub: https://github.com/pacaunt/typst-presentate
Compiled on 05 March 2026

Contents

1 Introduction 3
  1.1 Acknowledgements 3
2 Getting Started 4
  2.1 Creating slides 4
  2.2 Simple animation 4
  2.3 Control the show and hide: index specification 5
  2.4 Time traveling with relative indices 6
  2.5 Specific functions for `list` and `enum` 7
3 Advanced Usage 9
  3.1 Groupping pauses: the `fragments` function 9
  3.2 Change the hiding method: the `hider` argument 9
  3.3 Change the appearance of the content: the `transform` function 10
  3.4 Precise control of the animation: `motion` and `tag` function 10
  3.5 Advanced `motion`'s control command specification 13
4 Package Intregration Framework 13
  4.1 The `render` workspace 13
  4.2 `animation` module 14
  4.3 Animate the inanimate: a custom way to hide 15
5 Configuration Options 17
6 Exposed Utilities 17
  6.1 Internal states 17
7 Themes 17

1 Introduction

`Presentate` is a Typst package for creating *dynamic* PDF presentation that is compatible with other packages. The word *dynamic* means the compiled PDF contains *animated* content. But PDF is a static document format, how can it contains animations? Presentate will look into the content, and create a set of pages that reveal or hide some content based on the current number of frames called *subslides*, so that when going through the pages, it seems like the content is showing or hiding like a simple animation.

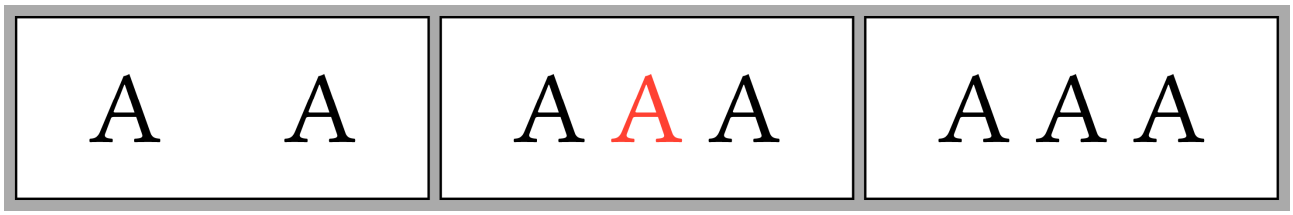
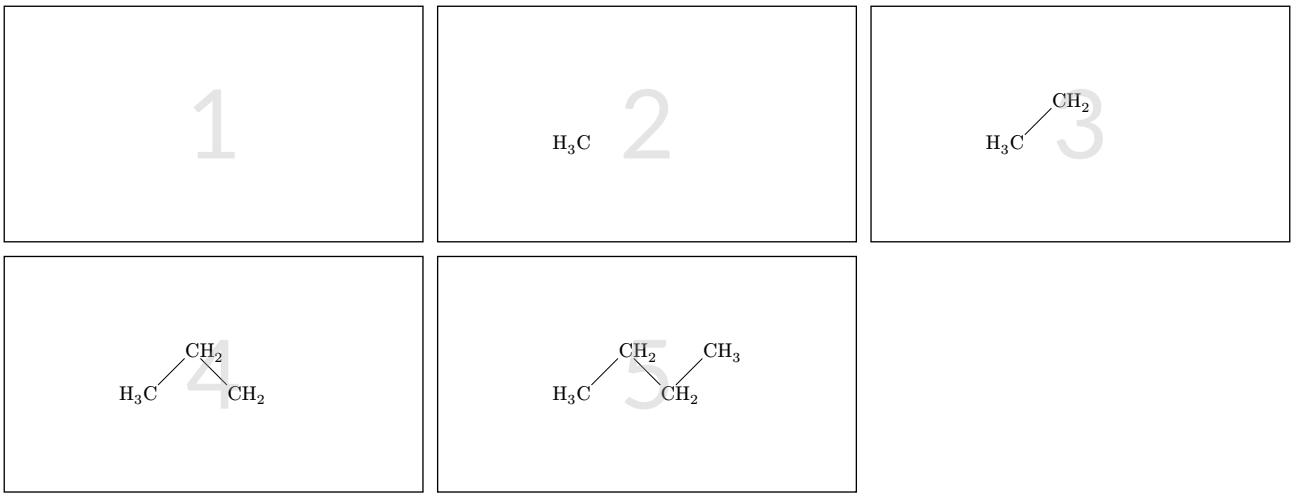


Figure 1: Example of fake animation in PDF format. Each page contains its own content and rules controlling them to hide, show, or decorated in some ways.

Dynamic PDF presentation packages have been in the Typst community for a long time, with the similar strategy for creating the animations, so why Presentate existed? The answer lies on the fact that presentation contains many kinds of visualization like plots, drawings, or chemistry molecules. These elements are usually created from other packages which have their own functions and data types. Therefore, to create animation with these visualization, the presentation framework must be able to handle *any* data types:

- be able to *hide* or *show* any elements, not only `content`,
- be able to generate the number of required subslides *automatically*.

Presentate is created for that purpose. It provides not only the slide animation engine, but also a framework and functions to interact with content from other packages. For example, here is some animation of drawing molecules integrated with the Alchemist[1] package.



Note that the gray text represents the subslide (frame) of that page.

1.1 Acknowledgements

The package was created by mixing my original motivation and insprations from many existing presentation packages. Thanks to:

- Polylux[2] for `subslide` implementation and pdfpc support,

[1] https://typst.app/universe/package/alchemist

- Touying[3] for idea of render frame, fake frozen states, and
- Minideck[4] for `only`, and `uncover` functions.
- Navigator[5] for the navigation system, progressive-outline, and the structured themes.

2 Getting Started

Import the package by

```
#import "@preview/presentate:0.2.5": *
```

Or download a zip file of Presentate's repository[6], install the `src` folder and import the package by

```
#import "src/export.typ": *
```

Then, setting your own styles, for example,

```
#set page(paper: "presentation-16-9") // you can change to 4-3
#set text(size: 25pt)
// your presentation begins here!
```

2.1 Creating slides

A slide can be created by using the `slide` function. For example,

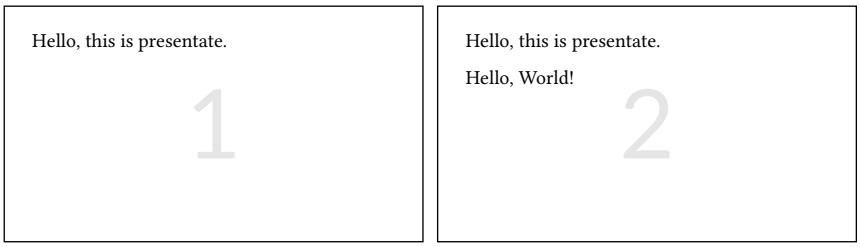
```
#slide[
  Hello, this is presentate.
]
```

2.2 Simple animation

In the slide function, you can use the animation functions to control the behavior of the content on the slide. The simplest animation is to show some content after another content, this can be done by `pause` function:

```
#slide[
  Hello, this is presentate. #show: pause

  Hello, World!
]
```

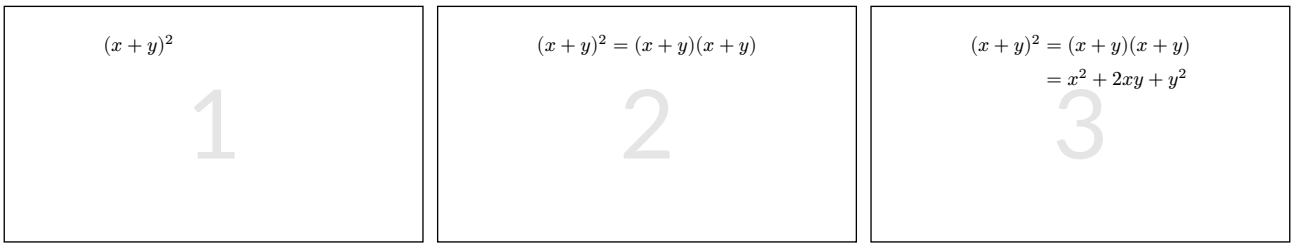
yields



[2] https://github.com/polylux-typ/polylux
[3] https://github.com/touying-typ/touying
[4] https://github.com/knuesel/typst-minideck
[5] https://github.com/eusebe/typst-navigator
[6] https://github.com/pacaunt/typst-presentate

Each content wrapped by the `pause` function will be revealed one by one on each frame. This function can be used in `math.equation` as well:

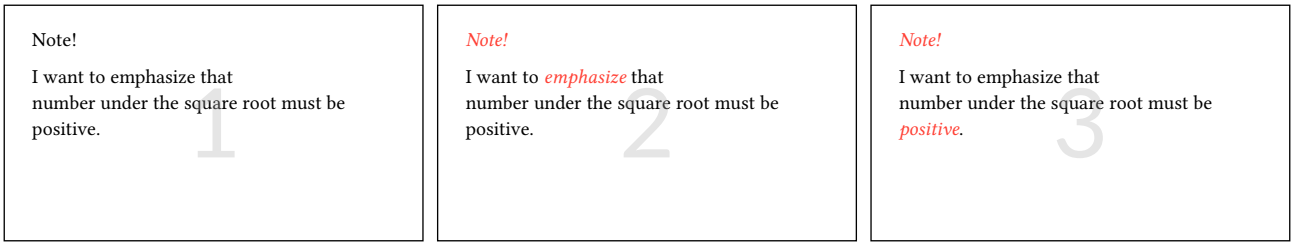
```
#slide[
  $
    (x + y)^2 pause(&= (x + y)(x + y)) \
              pause(&= x^2 + 2x y + y^2)
  $
]
```



2.3 Control the show and hide: index specification

If, for example, you want to emphasis a text on a slide, you can use the `alert` function. This function, by default, will *emphasize* the content by wrapping it in the typst's `emph` function and become *alerted* on the specified subslide.

```
#show emph: set text(fill: red) // for apparent looking
#slide[
  #alert(from: 2)[Note!] // stays alerted from subslide 2

  I want to #alert(2)[emphasize] that \
  number under the square root must be #alert(3)[positive].
]
```



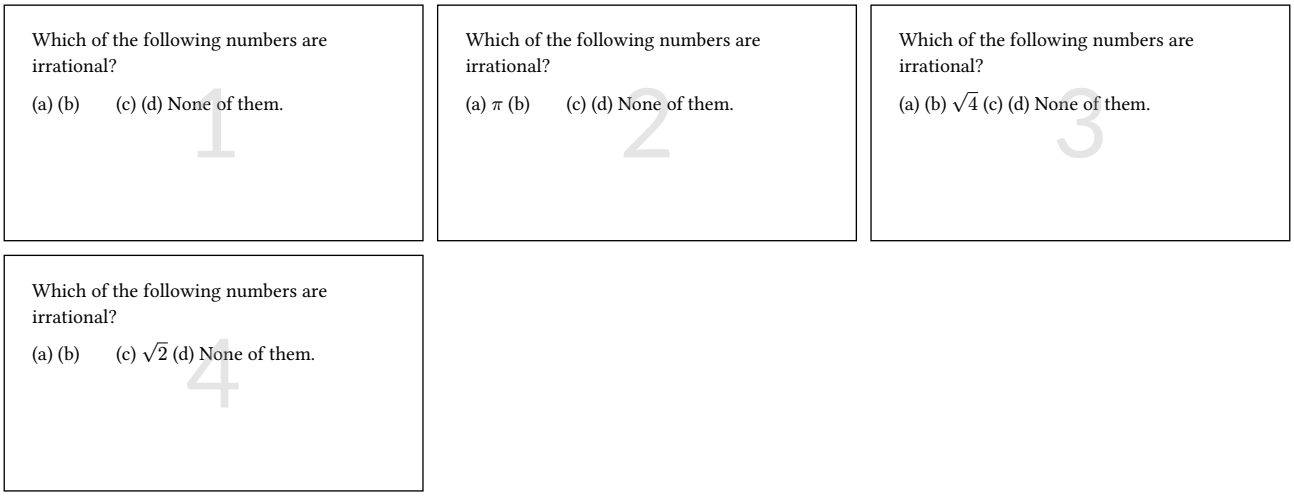
In the example, `alert` function accepted an integer and it made the text red only when the subslide number is the same as the specified number. This is called **index specification** where the index (pl. indices) is the subslide number or *frame* number. Indices can be specified in many ways:

- It can be an integer, to alert the content only at that subslide.
- It can be `from: int`, to start alerting since the `int`th subslide.
- It can be multiple integers like `alert(1, 4, 5, body)` to only alert on subslide 1, 4, and 5.
- It can be *both* integers and `from: int`, to show the modified content on the specified subslide and after the `int` subslide specified by the `from` argument.

Apart from `alert`, if you have ever used `beamer` in LaTeX before, you probably know the `uncover` and `only` functions. The `uncover` only *uncover* the content on the spcified subslides, and hide the content otherwise, with *space preserved*, while `only` function also works with the same logic, but its hiding method is to completely remove the content out, so no space is preserved.

```
#slide[
```

```
  Which of the following numbers are irrational?

  (a) #only(2, $pi$)
  (b) #uncover(3, $sqrt(4)$)  // The space here is peserved.
  (c) #only(4, $sqrt(2)$)
  (d) None of them.
]
```



2.4 Time traveling with relative indices

Most of the time when creating presentation, the step of revealing the content is either

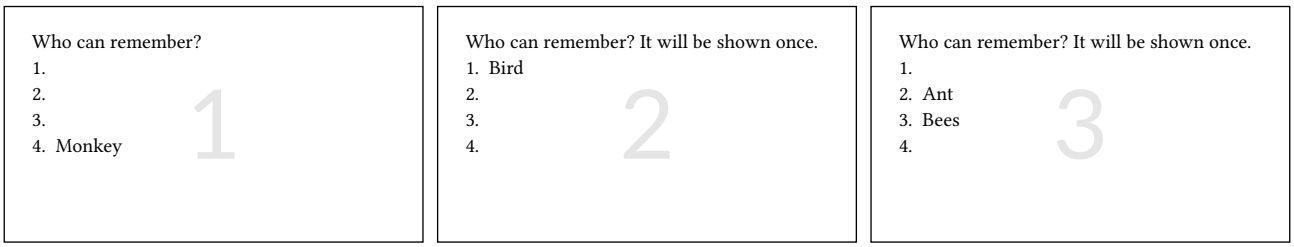
- showing at the same subslide as previous animation, or
- showing at the next subslide of the recent animation.

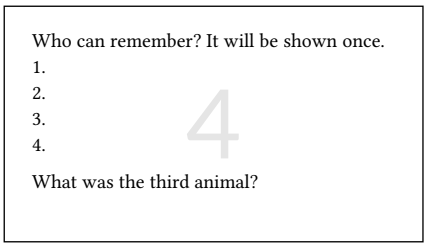
Presentate have a special type of index called **relative indices**, which are indices relative to the current number of pauses appear on the slide. There are 3 forms of relative indices:

1. `none` means the animation will happen at the same subslide as the current pause.
2. `auto` means the animation will happen at the *next* subslide after the current pause.
3. `(rel: int)` means the animation will happen at the `int` subslide away from the current pause.

For example,

```
#slide[
  Who can remember?
  #pause[It will be shown once.] // first pause
  + #uncover(none)[Bird] // with the pause
  + #uncover(auto)[Ant] // after the pause
  + #uncover((rel: 1))[Bees] // also after the pause
  + #uncover((rel: -1))[Monkey] // before the first pause
  #uncover(from: (rel: 2))[What was the third animal?]
]
```



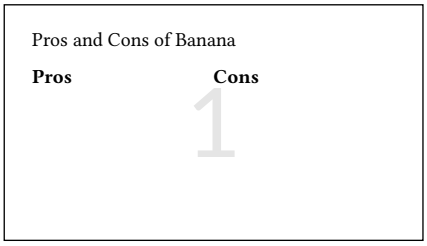


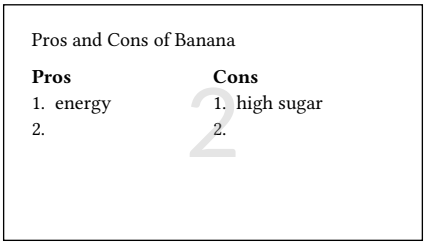
As you can see, the last text that was shown on subslide 4 was a little bit inconvenient to write `(rel: 2)` to make it shown when every animation was finished. It would be nice if we can use `pause` at the end and make it 'see' the previous `uncover`s. You can make the next pause reveal a content after (almost) any animation functions by specifying `update-pause: true` into its argument.

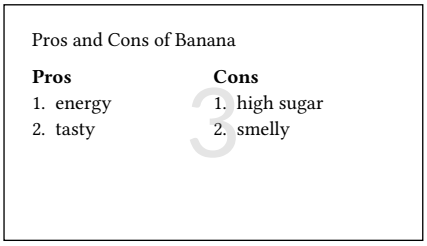
```
#slide[
    Who can remember?
  #pause[It will be shown once.] // first pause
  + #uncover(none)[Bird]
  + #uncover(auto)[Ant]
  + #uncover((rel: 1), update-pause: true)[Bees] // update the last animation
  + #uncover((rel: -1))[Monkey]
  #pause[What was the third animal?] // so this shows later
]
```

With this interaction between the current number of pauses and the `update-pause` argument, you can set the pauses to *any* number to modify the animation. For example, making the content shown in sync side by side:

```
#slide[
  Pros and Cons of Banana
  #grid(columns: (1fr,) * 2)[
    *Pros* #show: pause
    + energy
    #show: pause
    + tasty
  ][ // [] is the dummy content.
    #uncover(1, [], update-pause: true)
    *Cons* #show: pause
    + high sugar
    #show: pause
    + smelly
  ]
]
```





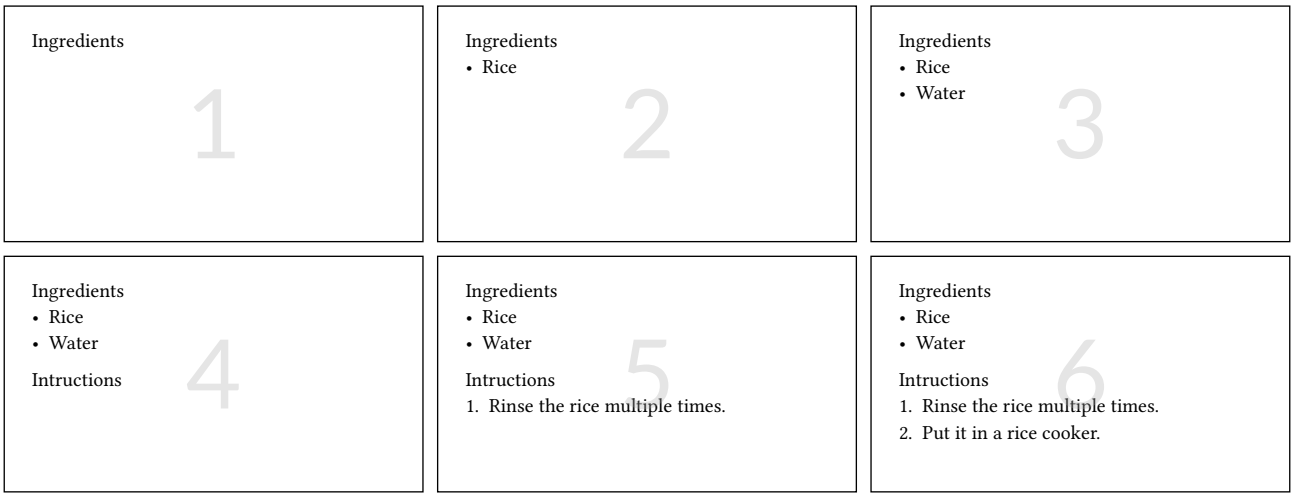


2.5 Specific functions for `list` and `enum`

In the last example, you can see that the number in `enum` did not hide properly. Since Typst treats the marker of `list` and number label of `enum` in different context from its body, Presentate implements

some (hacky) way to properly hide them without wiggling. To show the list/enum item by item, Presentate provides a function called `step-item`:

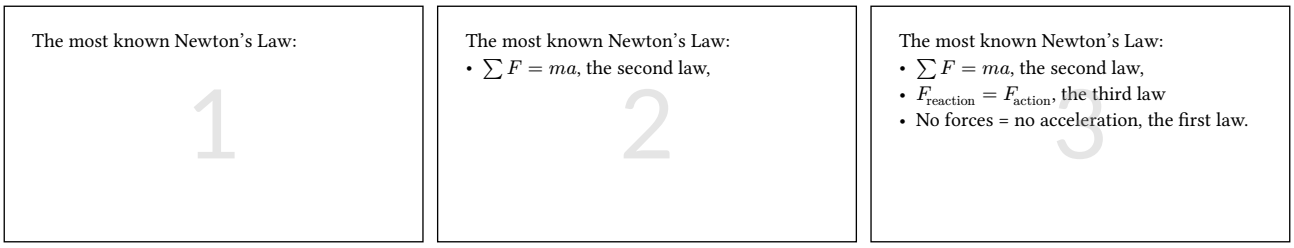
```
#slide[
  Ingredients
  #step-item[
    - Rice
    - Water
  ]
  #show: pause
  Intructions
  #step-item[
    + Rinse the rice multiple times.
    + Put it in a rice cooker.
  ]
]
```



Each of the item's body and its marker are wrapped in `pause` function, and synchronize the animation by using the `update-pause` hack. Therefore, you can nested them indefinitely, but it always update the pauses.

If you want to reveal the items chunk by chunk, Presentate also have the `reveal-item` function. This function can show item in group by group without breaking them. However, since the implementation of `reveal-item` requires additional Typst's state, it can decrease the speed of Typst compilation. So use this function sparingly.

```
#slide[
  The most known Newton's Law:
  #reveal-item[
    - $sum F = m a$, the second law,
  ][
    - $F_"reaction" = F_"action"$, the third law
    - No forces = no acceleration, the first law.
  ]
]
```

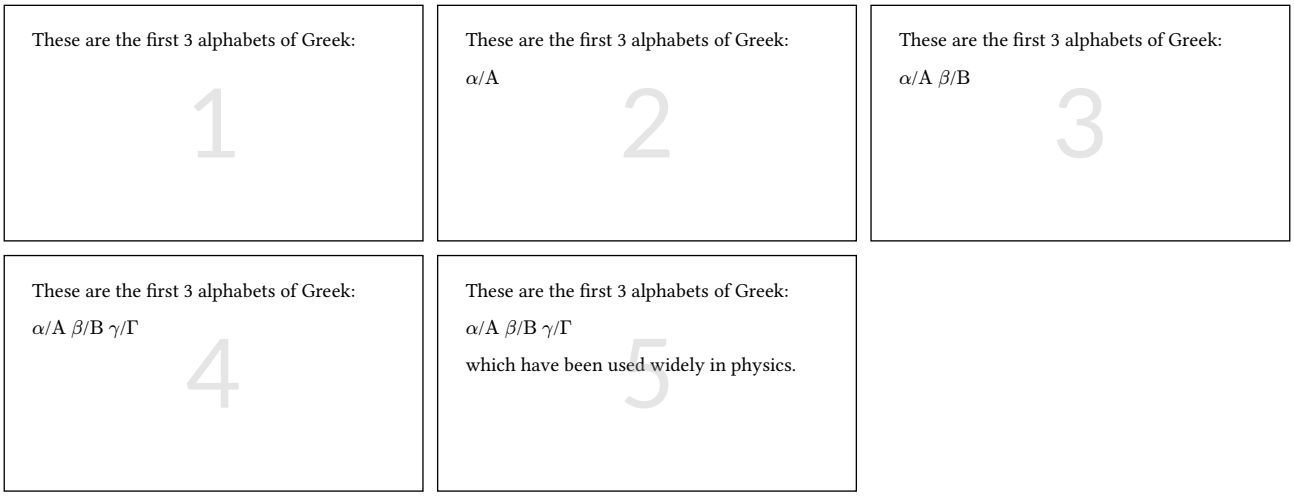


3 Advanced Usage

3.1 Groupping pauses: the `fragments` function

This function works like a shorthand for multiple pause/uncover calls. Moreover, you can choose whether to make it update the current pause by `update-pause` argument.

```
#slide[
  These are the first 3 alphabets of Greek:

  #fragments[$alpha$/$Alpha$ ][$beta$/$Beta$ ][$gamma$/$Gamma$]
  #show: pause

  which have been used widely in physics. // the fragments update the pause.
]
```



3.2 Change the hiding method: the `hider` argument

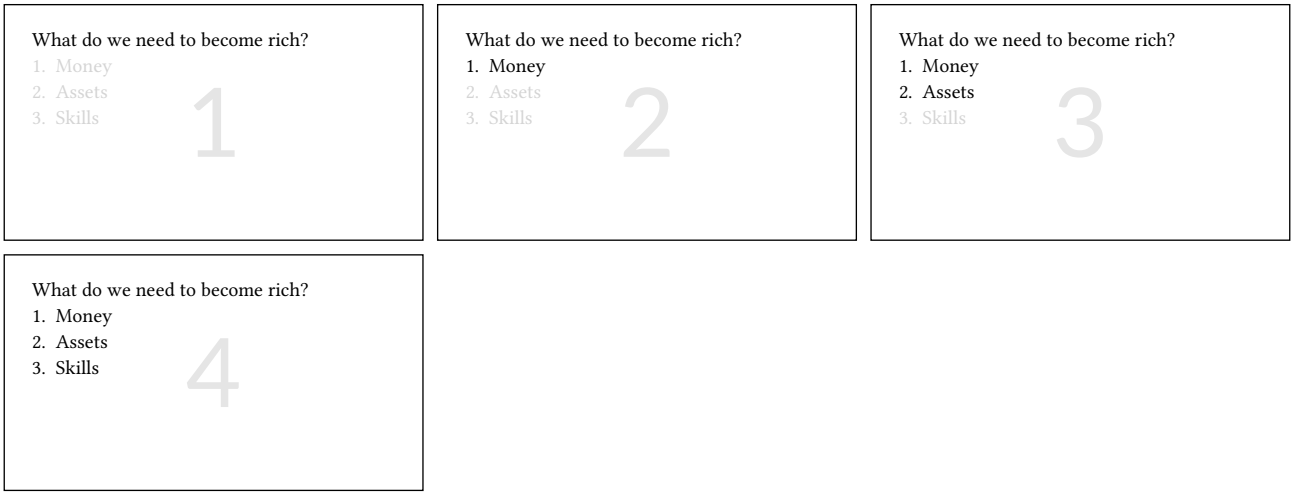
Every animation function in Presentation has a named argument `hider`, which is a function that used internally to *hide* the content. For example,

- `only` uses `it` ⇒ `none` as hider, which make the content disappear when it hides,
- `uncover`, `pause`, `step-item`, `fragments` uses native Typst's `hide` function, which preserves space.

If you want to change the hiding method of the text from completely hiding it to making it gray, you can use `text.with(fill: gray.transparentize(50%))` as the hider, like the following example,

```
#slide[
  What do we need to become rich?
  #step-item(hider: text.with(gray.transparentize(50%)))[
    + Money
    + Assets
    + Skills
```

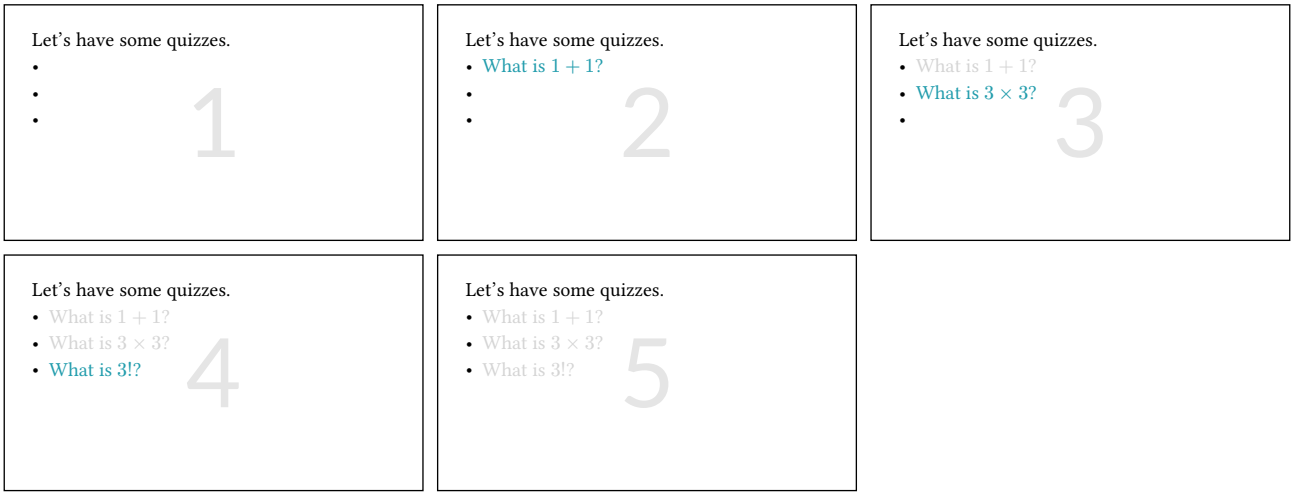
```
  ]
]
```



3.3 Change the appearance of the content: the `transform` function

`transform` requires a content as a base, it will be hidden by the `hider` until the specified index is reached (default is `auto`). Then, on each consecutive subslide, the content will transform by each function one by one. The following example shows how to emphasize content in a list by using `transform`.

```
#slide[
  #let yes = text.with(fill: eastern)
  #let no = text.with(fill: gray.transparentize(50%))
  Let's have some quizzes.
  - #transform([What is $1 + 1$?], yes, no)
  - #transform([What is $3 times 3$?], yes, no, start: none)
  - #transform([What is $3!$?], yes, no, start: none)
]
```



`start` argument tells when to start the animaton. Note that in this example, the grayed text was from `no` function, not the hider. The hider was still `hide` function, as you can see in the subslide 1.

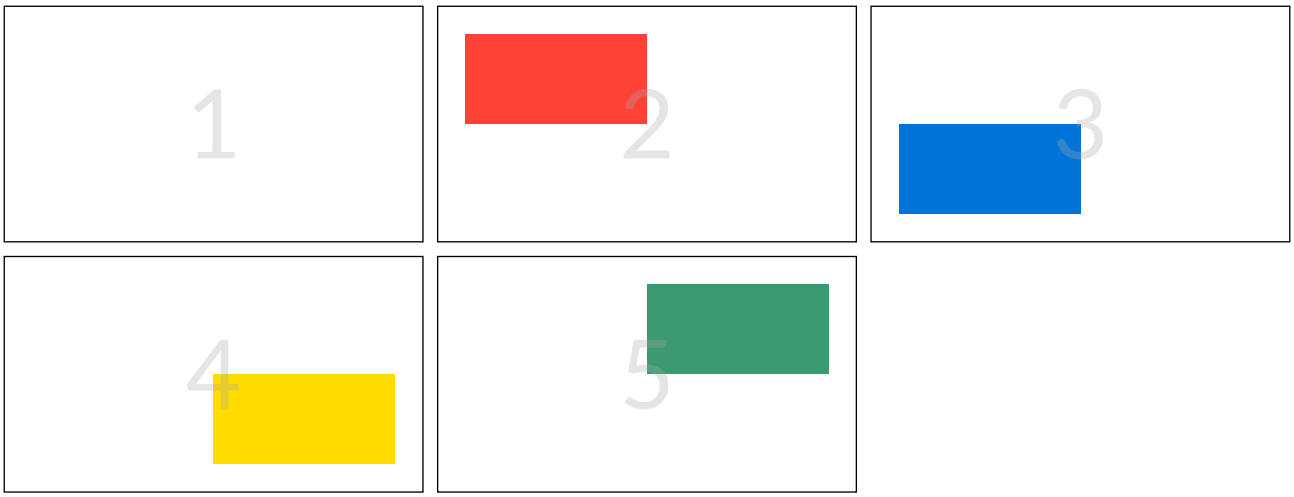
3.4 Precise control of the animation: `motion` and `tag` function

`motion` function provides a *workspace* for rendering content groups that are named by the `tag` function. The anatomy of this function is

```
#motion(s => [
  // some content and `tag(s, name, body)`
], controls: (
  // rules to show the `tag`
))
```

Motion function accepts only one positional argument which is a **function** that receives the presentate's state `s` and return a content block. `motion` function have a `controls` argument to specify which content group should be shown or hide precisely subslide by subslide. The motion reads the `controls` rules, process them and send the showing information through the state `s`. This state `s` is read by the `tag` function to show or hide the content accordingly.

```
#slide[
  #motion(s => {
    set rect(width: 100%, height: 100%)
    grid(
      rows: 1fr, columns: (1fr,1fr),
      tag(s, "red", rect(fill: red)),
      tag(s, "green", rect(fill: olive)),
      tag(s, "blue", rect(fill: blue)),
      tag(s, "yellow", rect(fill: yellow))
    )
  },
  controls: (),"red", "blue", "yellow", "green"))
]
```



This framework allows user to control the animation step without worrying about the location of each elements in the source code. For the specification of **controls** argument, here is the syntax:

- The `controls` only accept **an array of rules** that will be applied on each subslides, similar to `transform` and the functions.
- Each rule can be a single command or an array of commands that will be use at the same subslide.

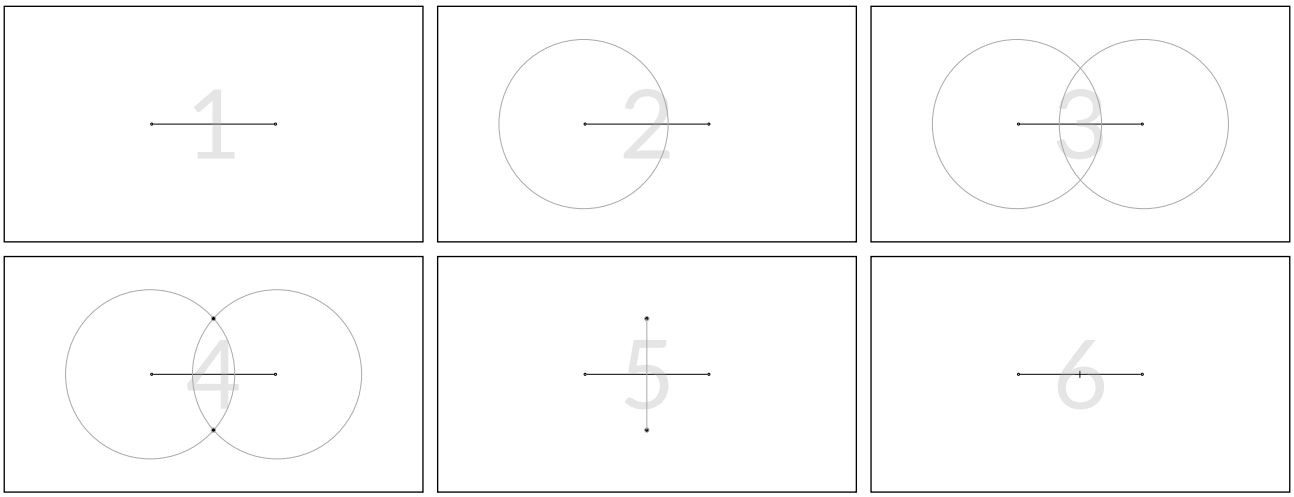
For each command,

- You can provide a name of the tag to show the content **once**,
- or provide with a suffix `.start` to start showing the content from the current step,
- or with a suffix `.stop` to stop showing the content.

The following example is more complex and use all of the control rule specification.

```
#import "@preview/cetz:0.4.2": canvas, draw
```

```
#slide[
  #set align(center + horizon)
  #motion(
    s => [
      #canvas({
        import draw: *
        scale(1.5)
        stroke((thickness: 2pt))
        line(name: "L", (0, 0), (6, 0), mark: (symbol: "o"))
        stroke((paint: gray))
        intersections("I", {
          tag(s, "circ1", circle("L.start", radius: 4))
          tag(s, "circ2", circle("L.end", radius: 4))
        })
        tag(s, "pt", {
          circle(radius: .1, "I.1", fill: black)
          circle(radius: .1, "I.0", fill: black)
        })
        tag(s, "cut", line("I.0", "I.1"))
        tag(s, "mark", {
          stroke(black)
          mark("L.mid", "L.end", symbol: "|", width: .5)
        })
      })
    ],
    hider: draw.hide.with(bounds: true),
    controls: (
      (), // don't show any group
      "circ1.start", // show the `circ1`
      "circ2.start", // show the `circ2`
      "pt.start", // show the `pt`
      ("circ1.stop", "circ2.stop", "cut"),
      // stop showing `circ1` and `circ2`, and show the `cut` once
      ("mark.start", "pt.stop"), // show the `mark` and stop showing the `pt`
    ),
  )
]
```



[7] https://typst.app/universe/package/cetz

This was created with integration of CeTZ[7] package, showing the capability to integrate with other packages. This integration works because of the `hider` argument in the motion function is the CeTZ's hide function, which is compatible for its own elements. Note that for more complex illustrations, you can provide custom `hider` to the `tag` function to use a specific hider for that group.

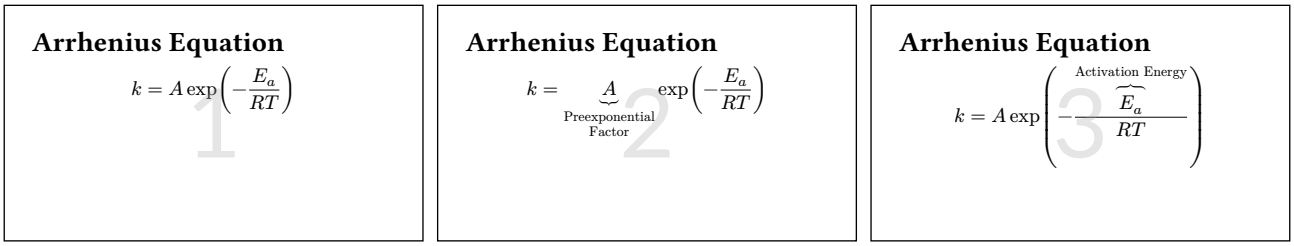
3.5 Advanced `motion`'s control command specification

For the version after 0.2.5, you can use a special syntax that let you modify the content in the tag group by applying some functions to it. The syntax of the command is

```
("{name}", {func}) // for apply the function once,
("{name}.start", {func}) // for starting showing the content and apply the function,
("{name}.apply", {func}) // for applying the function if it is showing,
"{name}.revert" // to revert back to original (identity) function
```

Here is an example of using this functionality: to annotate a mathematical expression:

```
#slide[
  = Arrhenius Equation
  #motion(
    s => [
      #let tag = tag.with(s) // a shorthand
      $ k = tag("A", A) exp(-tag("Ea", E_a)/(R T)) $
    ],
    controls: (
      ("A.start", "Ea.start", "exp.start"),
      ("A", it => $underbrace(it, "Preexponential\nFactor")$),
      ("Ea", it => $overbrace(it, "Activation Energy")$),
    ),
  )
]
```



4 Package Intregration Framework

Since each package has their own element and data types, to interact with them, Presentate must provide some functions that leave no trace to ensure there will be no conflict between packages. This functionality has been proposed in some ways before in the recent examples: changing the `hider` method of each animation functions. The later functions that will be shown use a similar logic; you have to specify how to *hide* the content from each packages *manually*. Moreover, since the functions leave no trace, Presentate cannot know what animation you have used, so you have to update each animation to Presentate *manually*.

4.1 The `render` workspace

`render` is a function that accepts another function that returns **an array of length two, containing the content and a state**. The syntax is as follows:

```
#render(s => ([
  #import animation: *
  // your content
], s))
```

where `s` is the **Presentate's state**, which is an array containing

- a dictionary of current presentation infomation, it is the first member of this array,
  - you can access the current subslide number by `s.at(0).subslide`,
  - the curent number of pauses by `s.at(0).pauses`, for example.
- the indices of the animation from each Presentate's animation functions.

The content inside `render` can be anything, but it needs to be a `content` data type. The state `s` here will be used by the functions in `animation` module, as will be presented in the next section.

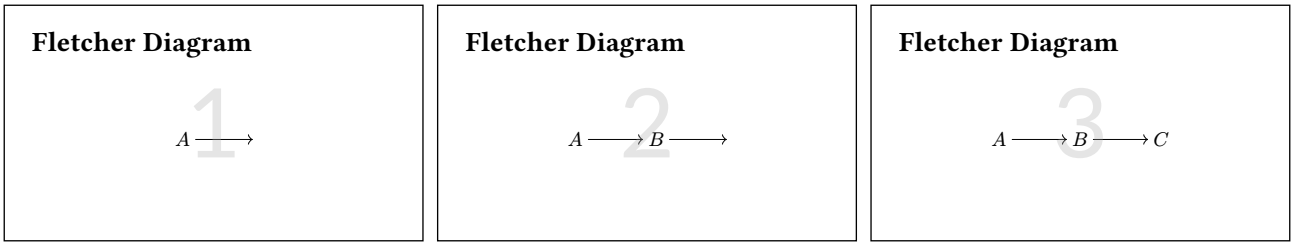
4.2 `animation` module

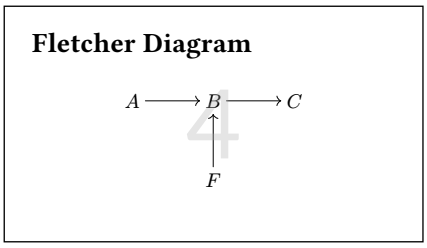
To fully use the animation in `render` with other packages, you can import the `animation` module inside the render call as shown in the last code listing. This module provides a similar set of functions like `pause`, `uncover`, `only`, and others, but all of them will be called a little bit different from the normal usage, as you have to put the state `s` **as the first positional argument** in the function *every time*, like

```
#render(s => ({
  import animation: *
  pause(s, { /* your content */  })
}, s))
```

Moreover, to make Presentate aware of the animation, you have to update the state `s` manually. Since `s` is an array, you can update by using `s.push(idx)` call, where `idx` is the indices that are required for the last animation to perform. For example, the `pause` function needs an additional new subslide, so we update `s` by `s.push(auto)` like in the following example.

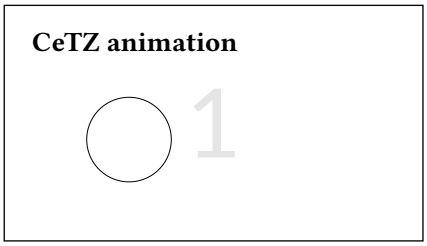
```
#import "@preview/fletcher:0.5.8": diagram, edge

#slide[
  = Fletcher Diagram
  #set align(center + horizon)
  #render(s => ({
    import animation: *
    diagram($
              A edge(→)  & pause(#s, B edge(→)) #s.push(auto)
                         & pause(#s, C) #s.push(auto) \
                         & pause(#s, F edge(→, "t"))
            $)
  }, s))
]
```

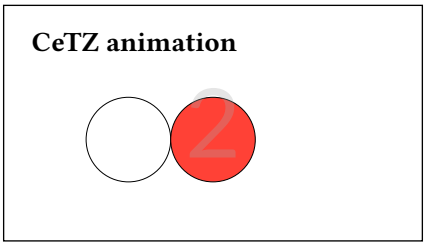


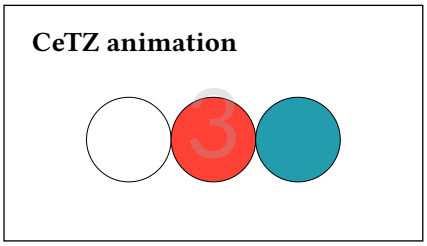


You can see that the diagram is jiggling, because by default, all functions in the `animation` module uses `it => none` as hider function. You can change it to suit the package's hide function, for example,

```
#import "@preview/cetz:0.4.2": canvas, draw
// create a custom pause function for CeTZ package's elements
#let cetz-pause = animation.pause.with(hider: draw.hide.with(bounds: true))
#slide[
  = CeTZ animation
  #set align(center + horizon)
  #render(s => ({
    import animation: *
    canvas({
      import draw: *
      scale(3)
      circle((0, 0))
      cetz-pause(s, circle((2, 0), fill: red))
      s.push(auto) // update the pause
      cetz-pause(s, circle((4, 0), fill: eastern))
    })
  }, s))
]
```







Not only you can specify `auto` as update indices, in fact, you can put

- an integer like `s.push(1)`, `s.push(2)` to **set** the current pause to that subslide number,
- relative indices like normal, but each index will **always** update the pause number,
- an *array* of any type of indices to prevent the pause update.

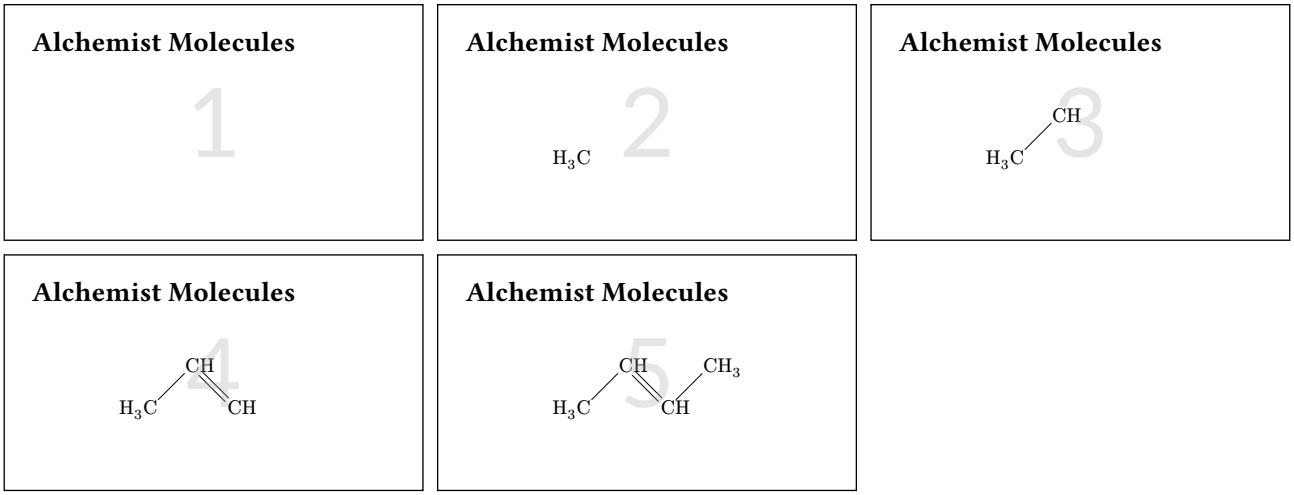
The specified indices are only for the current animation. For total number of frames needed to render all animation, Presentate will calculate it *automatically*.

4.3 Animate the inanimate: a custom way to hide

Some packages do not provide their `hide` function to hide their elements. Presentate also has a function for handle that case, which is the `animate` function in the `animation` module. This `animate` create another function that *react* with the current state `s`, so that the element will be shown one by one based on the current number of pauses, just like what normal pause do. To demonstrate this functionality, we will use an example to draw a chemical structure from Alchemist[8] package:

[8] https://typst.app/universe/package/alchemist/

```typ
#import "@preview/alchemist:0.1.8" as alc

#let modifier(func, ..args) = func(stroke: none, ..args) // set stroke to `none`
#let (single, double) = animation.animate(modifier: modifier, alc.single, alc.double)
#let (fragment,) = animation.animate(
  // set atom colors to white
  modifier: (func, ..args) => func(colors: (white,),..args),
  alc.fragment
)

#slide[
  = Alchemist Molecules
  #set align(center + horizon)
  #render(s => (
    {
      alc.skeletize({
        fragment(s, "H_3C")
        s.push(auto)
        single(s, angle: 1)
        fragment(s, "CH")
        s.push(auto)
        double(s, angle: -1, from: 0)
        fragment(s, "CH")
        s.push(auto)
        single(s, from: 0, angle: 1)
        fragment(s, "CH_3")
      })
    },
    s,
  ))
]

```



The animate function have a `modifier` argument that receives a function that will be called when the element is hidden. In this example, the single bond is hidden by setting its stroke to `none` and the atoms are hidden by setting their text color to white. The animate can process multiple functions at the same time if those functions require the same modifier to be hidden, so `animate` will returns **an array** of *modified* functions in the same order as specified, as shown when processing `alc.single` and `alc.double` with the same modifier.

5 Configuration Options

You can set the following options to Presentate:

- `handout: false` to create a handout version. This option will disable all of the animation and exposed all of the hidden content.
- `drafted: false` to show the current subslide number.
- `freeze-states: true` to freeze the states and counters. Note that this functionality was inherited from Touying package, and it will freeze only the specified states and counters, which can be set in `frozen-states-and-counters` option.
- `frozen-states-and-counters` to indicate which states and counters do Presentate need to freeze. The default is

```
(
  counter(figure.where(kind: image)),
  counter(figure.where(kind: table)),
  counter(footnote),
  counter(heading),
  counter(math.equation),
)
```

All of these options can be set by

```
#set-options(..options)
```

where `options` are the options in the form `key: value`.

6 Exposed Utilities

6.1 Internal states

Internally, Presentate store all of the slides information in a state in the `store` module. You can access this state by

```
context store.states.get()
```

The state's information will be an array of indices with the global info in the first member of this array. It is the same `s` as you can access from `render` or `motion` function.

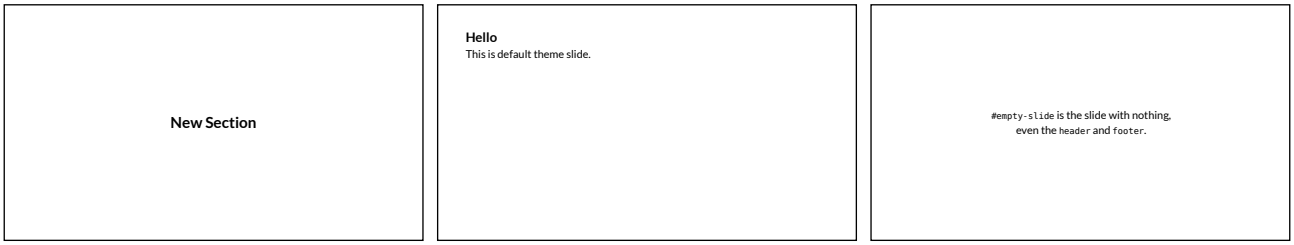
7 Themes

There are 2 types of themes that Presentate provides:

1. **normal type**, which is just a decoration of the slide,
   - **default** theme, just a normal, plain presentation,
   - **simple** theme, a decorative theme with auto heading continuation,
   - **classic** theme, a plain theme that slides can inherit the same title from the current level 2 heading and auto fit-to-height image insertion.
2. **structured themes**, which contains a functionality from Navigator[9] package. These amazing themes are created by David Hajage, which has a complete guide of how to use the themes in theme guide.

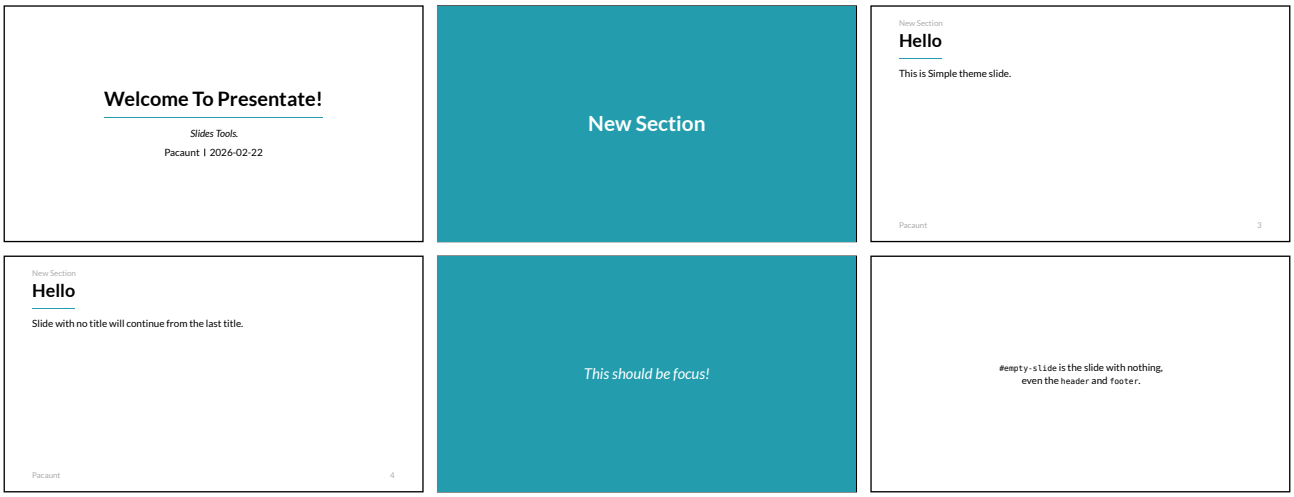
[9] https://typst.app/universe/package/navigator

Default Theme




```
#import themes.default: *

#show: template.with(
  aspect-ratio: "16-9"
)

= New Section

#slide[
  == Hello
  This is default theme slide.
]

#empty-slide[
  #set align(center + horizon)
  `#empty-slide` is the slide with nothing, \
  even the `header` and `footer`.
]
```

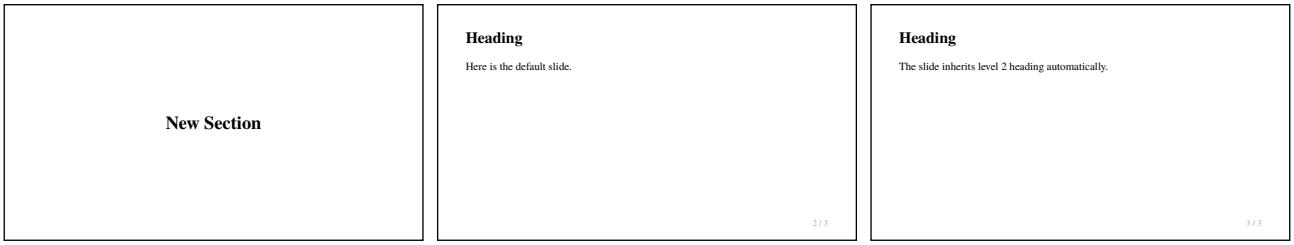
Simple Theme




```
#import themes.simple: *

#show: template.with(
  author: [Pacaunt], // change to yours!
  title: [Welcome To Presentate!],
  subtitle: [Slides Tools.],
)

= New Section

#slide[Hello][
```

```
  This is Simple theme slide.
]

#slide[
  Slide with no title will continue from the last title.
]

#focus-slide[
  This should be focus!
]

#empty-slide[
  #set align(center + horizon)
  `#empty-slide` is the slide with nothing, \
  even the `header` and `footer`.
]


```

Classic Theme




```
#import themes.classic: *

#show: template.with(
  aspect-ratio: "16-9",
)

= New Section

== Heading

#slide[
  Here is the default slide.
]

#slide[
  The slide inherits level 2 heading automatically.
]
```

Structured Themes

You can visit the examples of the structured themes here:

- minimal
- progressive-outline
- sidebar
- split
- miniframes